CSC-648-01-SU26
Project : Student Center
Team #4: BigBacks

Version History

Milestone 3 Version 1	6/30/2026
Milestone 3 Version 2	7/16/2026

Team Members

Wen Chien Yen	Team Lead / Scrum Master	wyen@sfsu.edu
Oscar Wang	Frontend Lead / Technical Writer	owang1@sfsu.edu
Banghao Yuan	Software Architect	Byuan1@sfsu.edu
Sean Wang	Database Administrator	swang29@sfsu.edu
Enoch Lin	Github Master	elin5@sfsu.edu
Kashif Zada	Backend Lead	kzada@sfsu.edu

Revision History

Date	Member	Task	Details
7/21/2026	Oscar Wang	Frontend structure	Revised the Homepage's format to include mentions of the application's unique features. Homepage now serves as a demo page as well as the main attention grabber of the new users. Also created the template for the degree progress report list. Sections are divided into Major, GE, and Electives.
7/20/2026	Sean Wang	UI & UX Wireframes	Added UI Wireframes for specific use cases.
7/20/2026	Kashif Zada	Frontend structure	Redesigned the Student Profile page into three grouped panels (Student Information, Academic Information, Contact Information), with inline editable forms for preferred name, walking speed, phone number, and address.
7/20/2026	Kashif Zada	Frontend structure	Added Cart page to application, allowing students to review, manage, and check conflicts/requireme

			nts for classes in their cart before enrolling.
7/20/26	Enoch Lin	Frontend Structure	Updated Home Page information
7/20/26	Sean Wang	Frontend Structure	Added Degree Planner page to application.
7/19/26	Banghao Yuan	Frontend Structure and Backend Architecture	Redesigned the original Planned Schedule into a weekly calendar interface that allows students to view both cart and enrolled classes by semester. Also implemented supporting backend APIs for schedule data, section details, time conflict detection, and walking-time warnings, while adding automated tests and documentation to verify the feature.
7/19/26	Banghao Yuan	Frontend Structure and Backend Architecture	Overhauled the Admin Portal with a modern sidebar-based interface and expanded its administrative tools for managing invitations, students, courses, class sections, departments, majors, degree programs, buildings, and classrooms. Also implemented dashboard statistics, role-based access

			control, supporting backend APIs and tests, and reorganized the Admin interface into smaller components to improve readability and maintainability
7/18/26	Banghao Yuan	Frontend Structure and Backend Architecture	Developed the Student Notifications feature with a dedicated interface for displaying academic reminders, enrollment updates, deadlines, conflicts, and system messages. Also implemented the supporting backend API, application routing, error and empty states, automated tests, and feature documentation.
7/18/26	Sean Wang	Frontend Structure	Added Degree Planner, Cart & Schedule UI
7/16/26	Banghao Yuan	Backend Architecture	Refactored the Planned Schedule backend to organize planned courses by course and semester rather than by individual class section. Also added support for adding, moving, listing, and removing planned courses, along with student access control, duplicate prevention, and validation for

			courses and semesters.
7/14/26	Wen Chien Yen	Project Management / Submission Coordination	Reviewed the Milestone 3 README requirements, assigned team responsibilities for technical documentation, horizontal prototype, and software review, and coordinated next-step workflow for the team. Tracked teammate progress, clarified prototype testing issues, and prepared communication for the instructor regarding prototype access, database configuration, and regrade clarification.
7/14/26	Wen Chien Yen	Deployment/Prototype Verification/Credentials README	Verified the deployed prototype on AWS EC2 and confirmed frontend-backend-database connectivity. Fixed deployment configuration issues by ensuring Nginx forwarded /api requests to the Express backend and confirming the backend connected to the correct better_student_center RDS database. Updated the credentials README with the correct database name, SSH tunnel instructions, EC2 access information,

			backend health check, and prototype testing instructions.
7/14/26	Oscar Wang	Document Formatting	Reviewed and formatted the document, as well as updated the table of contents.
7/13/26	Oscar Wang	Frontend Structure	Overhauled the UI skeleton to have a modern design, and designed the vertical prototype to rely on a navigation bar for easy site navigation for the user(s). Also set the foundation for conditional access for pages, specifically for only allowing access to pages for users that should be viewing them.
7/12/26	Oscar Wang	Frontend Structure	Changed the navigation structure to use React Router instead of mapping.
7/13/26	Sean Wang	Database Architecture/ Frontend Structure	Added Normalization section to DB design. Updated login/signup flow. Updated Course Search styling.
7/13/26	Banghao Yuan	Backend Architecture	Update the Backend Modules,Security,Validation and CorrectnessHandling,Scalability
7/12/26	Sean Wang	Frontend/Backend	Expanded class section to include Professor and contact information.

			Added Campus Map files to frontend. Made class search more compact, with each course collapsible and each section selectable. Added Campus Map to Course Search page.
7/12/26	Banghao Yuan	Backend Architecture	Update the UML Diagram with our current design and implementation.
7/11/26	Sean Wang	Database Architecture	Reused Data Definitions from M2. Revised list of High Level Functional Requirements based on feedback from the team. Added EER diagram that models all database tables of our application.

Table of Contents

Data Definitions

Our Student Center application uses structured academics, planning, campus navigation, and financial data to support student center academic planning.

User and Role

1. **User**
    - A User is any individual who can access the Student Center system. A User stores both personal information and account information used for authentication and role-based access.
    - Attributes:
        - user_id: Unique identifier for the user.
        - first_name: User's first name.
        - last_name: User's last name.
        - email: User's university email address.
        - phone_number: Optional contact phone number.
        - created_date: Date when the user record was created.
    - Relationship: A User has one User Account. A User may act as a Student, or Admin depending on the assigned role.
2. **Student**
    - A student is a User who has access to academic planning tools, including course search, planned schedule, calendar, degree progress, campus map, car, notifications, and financial information.
    - Attributes:
        - student_id: Unique identifier for the student.
        - account_id: Reference to the student's User Account.
        - major_id: Reference to the student's declared Major.
        - academic_level: Student level, such as freshman, sophomore, junior, senior, or graduate.
        - student_type: Student category, such as first-time, continuing, transfer, or international.
        - catalog_year: Academic catalog year used for degree requirement rules.
    - Relationship: A student may have many Planned Schedules. Cart records Enrollment records, Notification and Financial records. A student belongs to one declared Major for degree planning purposes.
3. **Admin**
    - An Admin is a User who has permission to manage academic and system data, including courses, class sections, buildings, classrooms, degree requirements, users, and student financial information.
    - Attributes
        - admin_id: Unique identifier for the admin.
        - account_id: Reference to the admin's User Account.
        - permission_level: Level or category of administrative access.

        - department_scope: Optional department or academic unit the admin manages.
    - Relationships: Admin users should only access admin functions through role-based authorization. Admin actions may affect Course, Class Section, Building, Classroom, Degree Requirement, and User data.
4. **Profile (UI-Generated Feature)**
    - A Profile is the user-facing display of a User's personal and academic information. It is not necessarily a separate database table; it may be generated from User, User Account, Student, and Major data.
    - Attribute:
        - display_name: User's full name.
        - email: University email address.
        - role: User role shown in the interface.
        - academic_level: Student academic level, if applicable.
        - student_type: Student type, if applicable.
        - declared_major: Student's declared major, if applicable.
        - contact_information: Editable phone or contact details.
    - Relationships: The Profile page should use data from the User, User Account, Student,, and Major records.
5. **Notification**
    - A Notification is a system-generated message that informs a User about important academic or system events.
    - Attribute
        - notification_id: Unique identifier for the notification.
        - user_id: Reference to the User receiving the notification.
        - notification_type: Type of notification, such as deadline, conflict, success, warning, or system message.
        - message: Notification text displayed to the user.
        - created_date: Date and time when the notification was created.
        - is_read: Indicates whether the user has read the notification.
        - dismissed_date: Date and time when the user dismissed the notification, if applicable.
    - Notifications may be generated by enrollment periods, approaching deadlines, schedule conflicts, insufficient walking time, or successful and failed system actions.

Academic Data

6. **Department**
    - A Department is an academic unit that offers Courses, manages Majors, and includes instructor names.
    - Attributes
        - department_id: Unique identifier for the department.
        - department_name: Official department name.
        - office_location: Department office location.
        - contact_email: Department contact email.
        - phone_number: Department phone number.
    - Relationship: A Department may offer many Courses, contain many Majors, and have many Instructors.
7. **Major**
    - A Major is a student's declared academic program. It defines the degree requirements used by the Degree Progress Tree.
    - Attributes
        - major_id: Unique identifier for the major.
        - department_id: Reference to the Department that owns the major.
        - major_name: Official major name.
        - required_units: Total units required for the major or degree.
        - catalog_year: Catalog year for the major requirements.
        - degree_type: Degree type, such as B.S., B.A., M.S., or M.A.
    - Relationship: A Major belongs to one Department and contains many Degree Requirements. A Student declares one Major for degree planning purposes.
8. **Degree Requirement**
    - A Degree Requirement is a requirement that a Student must complete for graduation. Requirements may include major required courses, major electives, general education categories, unit requirements, and prerequisite rules.
    - Attributes:
        - requirement_id: Unique identifier for the requirement.
        - major_id: Reference to the related Major.
        - requirement_category: Requirement type, such as major core, major elective, GE, prerequisite, or total units.
        - required_units: Number of units required.
        - requirement_description: Human-readable explanation of the requirement.
        - requirement_status: Status such as completed, planned, or remaining.
        - minimum_grade: Minimum grade needed to satisfy the requirement, if applicable.
    - Relationship: A Degree Requirement may be satisfied by one or more Courses. The Degree Progress Tree uses Degree Requirement data to show completed, planned, and remaining requirements
9. **Course**

- A Course is an academic subject offered by a Department. It describes the general course information and does not represent a specific scheduled class meeting.
- Attribute:
  - course_id: Unique identifier for the course.
  - department_id: Reference to the Department offering the course.
  - course_code: Official course code, such as CSC 648.
  - course_title: Course title.
  - units: Number of academic units.
  - description: Course description.
  - course_level: Level such as lower-division, upper-division, or graduate.
  - course_attributes: Attributes such as GE area, lecture, lab, or repeatable status.
  - prerequisites: Required courses or conditions before taking this course.
- Relationship:A Course may have many Class Sections across different Semesters. A Course may satisfy one or more Degree Requirements.

Enrollment and Class data

10. Class Section

- A Class Section is a specific scheduled offering of a Course in a Semester. It includes instructor name, meeting time, classroom, seat availability, and enrollment status.
- Attribute
  - section_id: Unique identifier for the class section.
  - course_id: Reference to the related Course.
  - semester_id: Reference to the Semester when the section is offered.
  - instructor_name: name of the Instructor teaching the section.
  - classroom_id: Reference to the assigned Classroom.
  - section_number: Official section number.
  - meeting_days: Days when the section meets.
  - start_time: Class start time.
  - end_time: Class end time.
  - modality: In-person, online, or hybrid.
  - meeting_type: Synchronous or asynchronous.
  - seat_capacity: Maximum number of students allowed.
  - seats_available: Number of remaining available seats.
  - waitlist_capacity: Maximum waitlist size.
  - waitlist_available: Number of remaining waitlist spots.
  - enrollment_status: Open, closed, waitlisted, or cancelled.
- A Class Section belongs to one Course and one Semester. It may appear in a Student's Planned Schedule, Cart, Calendar, or Enrollment Record.

11. Semester

- A Semester is an academic term in which Class Sections are offered and enrollment activity occurs.
- Attribute:
  - semester_id: Unique identifier for the semester.
  - term: Term name, such as Spring, Summer, or Fall.
  - year: Calendar year of the term.
  - start_date: Semester start date.
  - end_date: Semester end date.
  - add_drop_deadline: Last date for add/drop actions.
  - withdrawal_deadline: Last date for withdrawal.
  - enrollment_start_date: Date when enrollment begins.
  - enrollment_end_date: Date when enrollment ends.
- Relationship: A Semester contains many Class Sections. Planned Schedules, Carts, and Enrollment Records are associated with a Semester.

12. Enrollment Information

- Enrollment Information describes when and how a Student is allowed to enroll, add, drop, or withdraw from Class Sections for a specific Semester.
- Attributes:
  - enrollment_info_id: Unique identifier for enrollment information.
  - semester_id: Reference to the related Semester.

        - student_type: Student type affected by the enrollment window.
        - enrollment_start_time: Start time for enrollment access.
        - enrollment_end_time: End time for enrollment access.
        - add_drop_deadline: Deadline for adding or dropping classes.
        - withdrawal_deadline: Deadline for withdrawing from classes.
    - Relationships: Enrollment Information is used to determine whether a Student can perform enrollment actions during a given term.

13. **Enrollment Record**
    - An Enrollment Record stores a Student's actual enrollment history or current enrollment in a Class Section.
    - Attributes:
        - enrollment_id: Unique identifier for the enrollment record.
        - student_id: Reference to the Student.
        - section_id: Reference to the Class Section.
        - semester_id: Reference to the Semester.
        - enrollment_status: Status such as enrolled, dropped, withdrawn, completed, or waitlisted.
        - enrolled_date: Date when the student enrolled.
        - dropped_date: Date when the student dropped the section, if applicable.
        - grade: Final grade, if available.
    - Relationships: Enrollment Records represent official or sample academic history. They are different from Planned Schedule and Cart data because they represent actual enrollment rather than planning intent.

Schedule and Planning Data

14. Planned Schedule

- A Planned Schedule is a collection of Class Sections that a Student is considering or planning to take for a Semester.
- Attribute:
  - planned_schedule_id: Unique identifier for the planned schedule.
  - student_id: Reference to the Student who owns the planned schedule.
  - semester_id: Reference to the Semester being planned.
  - created_date: Date when the planned schedule was created.
  - last_updated: Date when the planned schedule was last modified.
  - total_units: Total units from selected Class Sections.
  - schedule_status: Status such as draft, active, or archived.
- Relationships :A Planned Schedule may contain many Class Sections. It supports time conflict detection, prerequisite checking, walking time calculation, and Calendar display. The Planned Schedule may also support basic walking time checking between selected Class Sections when building information is available.
-

15. Calendar (UI-Generated Feature)

- A Calendar is a weekly or monthly view of a Student's planned and enrolled Class Sections. It is primarily a user interface view generated from Planned Schedule and Enrollment Record data.
- Attributes:
  - calendar_view_type: Weekly or monthly view.
  - student_id: Reference to the Student.
  - semester_id: Reference to the Semester.
  - displayed_sections: Planned and enrolled Class Sections shown on the calendar.
  - selected_date_range: Date range being displayed.
- Relationships : The Calendar should display course code, meeting time,, building, and room. It should visually distinguish planned classes from enrolled classes.

16. Cart

- A Cart is a collection of Class Sections selected by a Student before final enrollment. It allows the Student to review selected sections, total units, enrollment status, and possible conflicts before enrolling.
- Attributes:
  - cart_id: Unique identifier for the cart.
  - student_id: Reference to the Student who owns the cart.
  - semester_id: Reference to the Semester.
  - created_date: Date when the cart was created.
  - last_updated: Date when the cart was last modified.
  - total_units: Total units in the cart.
  - cart_status: Status such as active, submitted, or archived.

- Relationships: A Cart may contain many Class Sections. A Cart is different from a Planned Schedule because it is closer to enrollment action.

17. Degree Progress Tree (UI-Generated Feature)

- A Degree Progress Tree is a visual academic roadmap that shows a Student's completed, planned, and remaining Degree Requirements.
- Attributes:
  - student_id: Reference to the Student.
  - major_id: Reference to the Student's declared Major.
  - completed_requirements: Requirements already satisfied by completed courses.
  - planned_requirements: Requirements expected to be satisfied by planned courses.
  - remaining_requirements: Requirements not yet satisfied.
  - remaining_units: Units still needed for graduation.
  - progress_percentage: Estimated percentage of degree completion.
- Relationships:The Degree Progress Tree is generated from Student, Major, Degree Requirement, Course, Enrollment Record, and Planned Schedule data. It helps Students understand how selected and completed courses connect to graduation requirements.

18. Course Details (UI-Generated Feature)

- Course Details is a user-facing view that combines Course information with available Class Sections, instructor information, schedule information, location, prerequisites, and seat availability.
- Attributes:
  - course_id: Reference to the Course.
  - course_code: Official course code.
  - course_title: Course title.
  - description: Course description.
  - units: Course units.
  - prerequisites: Required courses or conditions.
  - available_sections: List of related Class Sections.
  - instructor_information: Instructor name and office hour details.
  - location_information: Building and classroom information.
  - seat_availability: Seats available for each section.
- Relationships:Course Details may not need to be a separate database table. It can be generated by joining Course, Class Section, Classroom, Building, and Semester data.

19. Course Suggestion

- A Course Suggestion is a planning recommendation for Students who are close to graduation and may want to take advanced courses for graduate school preparation or career development.
- Attributes:
  - suggestion_id: Unique identifier for the suggestion.
  - student_id: Reference to the Student receiving the suggestion.
  - course_id: Reference to the suggested Course.
  - reason: Explanation of why the course is suggested.

        - suggestion_type: Type such as graduate preparation, career preparation, or major elective.
        - confidence_level: Optional value showing strength of the suggestion.
        - created_date: Date when the suggestion was generated.
    - Relationships: Course Suggestions are planning aids only. They should not be presented as official academic advising or guaranteed graduate credit approval.

Campus Navigation Data

20. Campus Map (UI-Generated Feature)

- A Campus Map is a map-based interface that displays campus buildings, classroom locations, department locations, and campus resources.
- Attribute:
  - map_name: Name of the campus map.
  - displayed_buildings: Buildings shown on the map.
  - displayed_resources: Campus resources shown on the map.
- Relationships: The Campus Map is mainly a user interface feature. It uses Building and Classroom data to highlight locations for selected Class Sections.

21. Building

- A Building is a physical campus structure where classes, departments, offices, or resources may be located.
- Attributes:
  - building_id: Unique identifier for the building.
  - building_name: Official building name.
  - address: Optional campus address or description.
- Relationships:A Building may contain many Classrooms and may be displayed on the Campus Map.

22. Classroom

- A Classroom is a room inside a Building where a Class Section may meet.
- Attributes:
  - classroom_id: Unique identifier for the classroom.
  - building_id: Reference to the Building.
  - room_number: Official room number.
  - room_capacity: Number of seats available in the room.
  - classroom_type: Lecture room, lab, seminar room, or other type.
- Relationships:A Classroom belongs to one Building. A Class Section may be assigned to one Classroom

Financial Data

23. Billing Profile

- A Billing Profile is a Student's financial account profile in the system. It connects the Student to charges, payments, refunds, financial aid credits, and outstanding balance information.
- Attribute:
  - billing_profile_id: Unique identifier for the billing profile.
  - student_id: Reference to the Student.
  - billing_status: Current billing status.
  - billing_email: Email used for billing contact.
  - billing_phone: Phone number used for billing contact.
  - last_updated: Date when billing information was last updated.
  - current_balance: Current outstanding balance.
- Relationships: A Billing Profile may have many Student Charges, Transactions, and Financial Aid records.

24. Student Charge

- A Student Charge is a financial charge assigned to a Student for a specific Semester or service.
- Attribute:
  - charge_id: Unique identifier for the charge.
  - billing_profile_id: Reference to the Billing Profile.
  - semester_id: Reference to the related Semester, if applicable.
  - charge_type: Type of charge, such as tuition, fee, housing, or other.
  - amount: Charge amount.
  - due_date: Payment due date.
  - payment_status: Status such as unpaid, partially paid, paid, or overdue.
- Relationships: Student Charges contribute to the Billing Profile's outstanding balance.

25. **Financial Aid**

- Financial Aid is aid information associated with a Student, such as loans, grants, scholarships, or awards.
- Attributes:
  - financial_aid_id: Unique identifier for the financial aid record.
  - student_id: Reference to the Student.
  - billing_profile_id: Reference to the Billing Profile.
  - aid_name: Name of the aid.
  - aid_type: Type such as loan, grant, scholarship, or award.
  - semester_id: Semester when the aid applies.
  - amount: Aid amount.
  - aid_status: Status such as pending, accepted, declined, or applied.
  - application_link: Link to the official financial aid application website, if applicable.
- Relationships: Financial Aid may reduce the Student's outstanding balance when applied. Financial aid application links should redirect students to official university or external financial aid systems when appropriate

26. Transaction

- A Transaction is a financial activity related to a Student's Billing Profile, such as a payment, refund, charge adjustment, or financial aid credit.
- Attributes:
  - transaction_id: Unique identifier for the transaction.
  - billing_profile_id: Reference to the Billing Profile.
  - transaction_type: Payment, refund, charge, or financial aid credit.
  - amount: Transaction amount.
  - transaction_date: Date of the transaction.
  - transaction_status: Status such as pending, completed, failed, or cancelled.
  - description: Description of the transaction.
  -
- Relationships: Transactions affect the current balance shown in the Billing Profile.

Prioritized High Level Functional Requirements

Priority 1:

Authentication / User

- FR 1.1: A User shall be able to log in with an email and password for the prototype.
- FR 1.2: A User shall be able to log out of their account.
- FR 1.3: A User shall be identified as either a Student or an Admin.
- FR 1.4: A User shall be shown features based on their assigned role.

Student Profile

- FR 2.1: A Student shall have an academic level, such as freshman, sophomore, junior, senior, or graduate.
- FR 2.2: A Student shall have a student type, such as first-time student, continuing student, transfer student, or international student.
- FR 2.3: A Student shall be able to view and update their profile.

Course Search

- FR 2.4: A Student shall be able to search for courses.
- FR 2.5: A Student shall be able to filter courses.
- FR 2.6: A Student shall be able to view course details.
- FR 5.1: A Course shall have basic course information, including course code, title, and units.
- FR 5.2: A Course shall be associated with a Department.
- FR 5.3: A Course shall have a detailed course description.
- FR 5.7: A Course shall have available Class Sections for a selected term.
- FR 5.8: A Course shall be searchable through class search.
- FR 5.9: A Course shall be filterable through class search.

Class Section

- FR 6.1: A Class Section shall have section details, including section number, term, instructor name, meeting time, location, enrollment status, and seat availability.
- FR 6.2: A Class Section shall be addable or removable from a Student's Planned Schedule.
- FR 6.3: A Class Section shall have a modality, such as in-person, online, or hybrid.
- FR 6.4: A Class Section shall have a meeting type, such as synchronous or asynchronous.
- FR 6.7: A Class Section shall indicate whether it conflicts with Class Sections already in a Student's Planned Schedule.

Planned Schedule

- FR 2.8: A Student shall be able to add or remove courses from a Planned Schedule.
- FR 2.11: A Student shall be able to view their Planned Schedule.
- FR 7.1: A Planned Schedule shall store all Class Sections selected by a Student.
- FR 7.2: A Planned Schedule shall display the relevant course information for each selected Class Section, including course code, section number, meeting time, location, modality, instructor name, and enrollment status.
- FR 7.3: A Planned Schedule shall allow a Student to add or remove Class Sections.
- FR 7.4: A Planned Schedule shall detect time conflicts between selected Class Sections.

Semester

- FR 13.1: A Semester shall show term name, start date, end date, add/drop deadline, withdrawal deadline, and enrollment period.

Department

- FR 14.1: A Department shall have a department name, office location, and contact information.
- FR 14.2: A Department shall show a list of Courses.

Notifications

- FR 2.23: A Student shall be able to view and dismiss Notifications.
- FR 19.1: A Notification shall inform a Student when their enrollment period begins.
- FR 19.2: A Notification shall inform a Student about approaching deadlines.
- FR 19.3: A Notification shall warn a Student about schedule conflicts or insufficient walking time between classes.
- FR 19.4: A Notification shall inform a Student when an attempted action succeeds or fails.

Priority 2:

Schedule

- FR 20.1: A Schedule shall display a Student's planned and enrolled Class Sections in a weekly or monthly format.
- FR 20.2: A Schedule shall display each Class Section on the correct day and time.
- FR 20.3: A Schedule shall visually distinguish planned classes from enrolled classes.
- FR 20.4: A Schedule shall display relevant course information, including course code, meeting time, instructor name, building, and room.
- FR 20.5: A Schedule shall allow a Student to select a Class Section and view its Course Details.
- FR 20.6: A Schedule shall allow a Student to select a class location and open it on the Campus Map.

Cart

- FR 2.24: A Student shall be able to view their Cart.
- FR 2.25: A Student shall be able to add or remove Class Sections from their Cart.
- FR 6.5: A Class Section shall be addable to a Student's Cart.
- FR 6.6: A Class Section shall indicate whether it is eligible to be added to a Cart.
- FR 21.1: A Cart shall store Class Sections selected by a Student.
- FR 21.2: A Cart shall display relevant course information, including course name, course number, units, instructor name, meeting time, location, and enrollment status for each section.
- FR 21.3: A Cart shall allow a Student to add or remove Class Sections.
- FR 21.4: A Cart shall calculate the total number of course units.

Degree Progress

- FR 2.12: A Student shall be able to view a Degree Progress based on their declared Major.
- FR 2.13: A Student shall be able to update their declared Major for major planning purposes.
- FR 2.14: A Student shall be able to view completed, planned, and remaining degree requirements.
- FR 2.15: A Student shall be able to view missing prerequisites for planned courses.
- FR 7.5: A Planned Schedule shall identify missing prerequisites for selected Class Sections.
- FR 7.6: A Planned Schedule shall support basic walking time checking between selected Class Sections when building information is available.
- FR 7.7: A Planned Schedule shall provide access to the Calendar view.
- FR 9.1: A Degree Progress shall display required General Education courses, major electives, and major requirements.
- FR 9.2: A Degree Progress shall display completed requirements, planned requirements, and remaining requirements.
- FR 9.3: A Degree Progress shall display remaining units needed for graduation.

- FR 9.4: A Degree Progress shall display prerequisite information for planned courses.
- FR 9.5: A Degree Progress shall recommend courses that satisfy remaining requirements.
- FR 9.6: A Degree Progress shall allow a Student to view which Courses can satisfy remaining requirements.
- FR 9.7: A Degree Progress shall update when a Student updates their Planned Schedule.
- FR 9.8: A Degree Progress shall update when a Student changes their declared Major.
- FR 9.9: A Degree Progress shall display which requirement category a planned Course satisfies.

Admin Management

- FR 3.1: An Admin shall be able to add, remove, or update Course information.
- FR 3.2: An Admin shall be able to add, remove, or update Class Section information.
- FR 3.3: An Admin shall be able to add, remove, or update Building location information.
- FR 3.4: An Admin shall be able to add, remove, or update Degree Requirement information.
- FR 3.6: An Admin shall be able to modify Student profile information.
- FR 15.1: A Major shall have a major name, related Department, required units, and catalog year.
- FR 15.2: A Major shall show the Degree Requirements.
- FR 16.1: A Degree Requirement shall have a requirement category, required units, and requirement status.
- FR 16.2: A Degree Requirement shall be satisfied by Courses.
- FR 16.3: A Degree Requirement shall indicate whether the requirement is completed, planned, or remaining.

Campus Map

- FR 2.16: A Student shall be able to view class building locations on the Campus Map.
- FR 8.1: A Campus Map shall display campus building locations.
- FR 8.2: A Campus Map shall display the building location for a selected Class Section.
- FR 8.3: A Campus Map shall display classroom location information for a selected Class Section.
- FR 8.4: A Campus Map shall display campus resources, such as library, gym, student center, HSS buildin…
- FR 8.6: A Campus Map shall allow a Student to return to the Calendar after viewing a class location.

Priority 3:

Enrollment Information and Enrollment Records

- FR 2.7: A Student shall be able to view enrollment information for the current term.
- FR 2.9: A Student shall be able to enroll in, add, drop, or withdraw from Courses when enrollment is open for their student group.
- FR 2.22: A Student shall be able to view Enrollment Records from different semesters.
- FR 4.1: Enrollment Information shall display the date and time when the Student's student type is eligible to begin enrollment.
- FR 4.2: Enrollment Information shall display the enrollment window for the current term.
- FR 4.3: Enrollment Information shall notify a Student when enrollment is available for their student type.
- FR 4.4: Enrollment Information shall display the add/drop deadline for the current term.
- FR 4.5: Enrollment Information shall display the withdrawal deadline for the current term.
- FR 10.1: An Enrollment Record shall display a Student's academic history across semesters.
- FR 10.2: An Enrollment Record shall display the Semester associated with each Course.
- FR 10.3: An Enrollment Record shall display the grade associated with each Course.

Financial Data

- FR 2.17: A Student shall be able to view outstanding charges.
- FR 2.18: A Student shall be redirected to the appropriate payment portal to make payments for outstanding charges.
- FR 2.19: A Student shall be redirected to the financial aid website for application.
- FR 2.20: A Student shall be able to view their financial aid awards.
- FR 2.21: A Student shall be able to accept or decline their financial aid awards.
- FR 3.5: An Admin shall be able to add, remove, or update student financial information.
- FR 11.1: Student Charges shall display charge information, including charge type, amount, and due date.
- FR 11.2: Student Charges shall display the total outstanding balance.
- FR 11.3: Student Charges shall display the estimated amount owed or refund amount after financial aid is applied.
- FR 11.4: Student Charges shall provide a link to the university payment portal for payment processing.
- FR 12.1: Financial Aid shall display name, type, term, amount, and status.
- FR 12.2: Financial Aid shall be applied toward outstanding charges when applicable.
- FR 12.3: Financial Aid shall provide a link or redirect to the university financial aid application website.
- FR 17.1: A Billing Profile shall be provided to Students.

- FR 17.2: A Billing Profile shall store billing contact information and payment-related account information.
- FR 17.3: A Billing Profile shall display all outstanding charges, payments, refunds, and financial aid credits.
- FR 17.4: A Billing Profile shall calculate the current outstanding balance.

UI and UX Wireframes

[Link to Figma Prototype Pages]

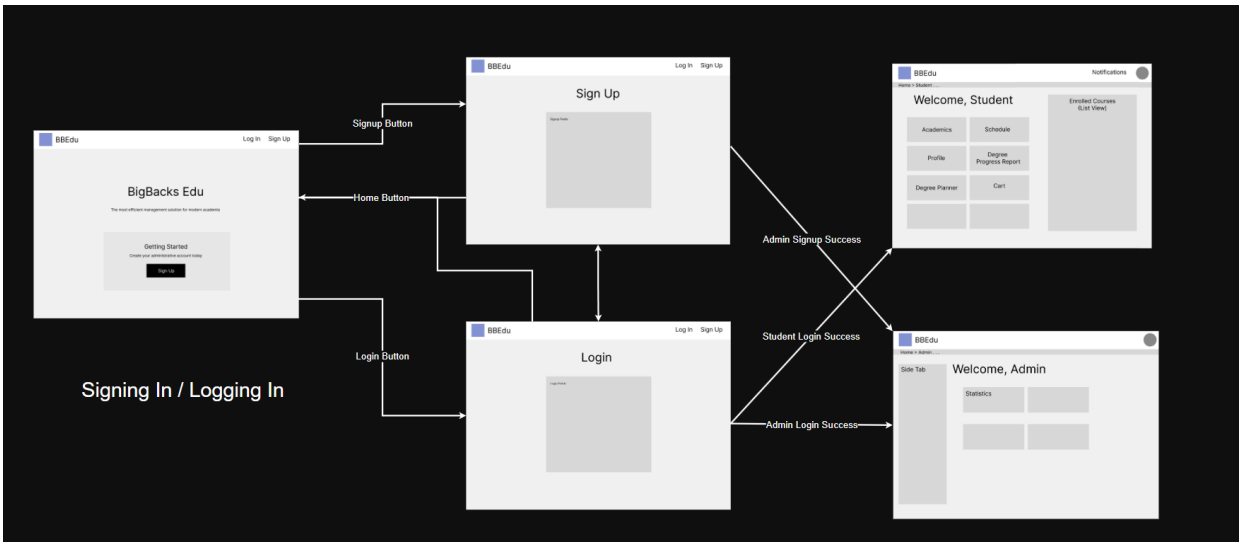
To access, please use the password:
`Summer2026Acorn`

[Link to draw.io Wireframes/Wireflows]

The UI/UX wireframes and wireflows are planning references and may not match the final application exactly, since the design and functionality may change during development.

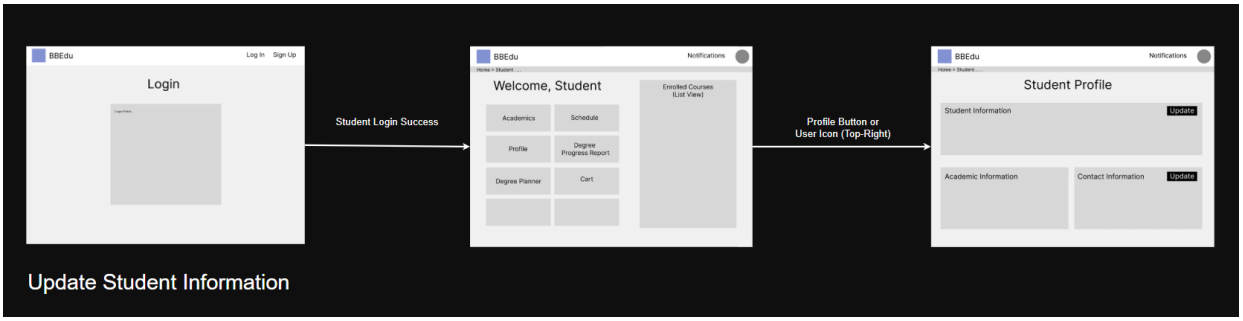
Use Cases

1: Signing In/Logging In (Student/Admin)



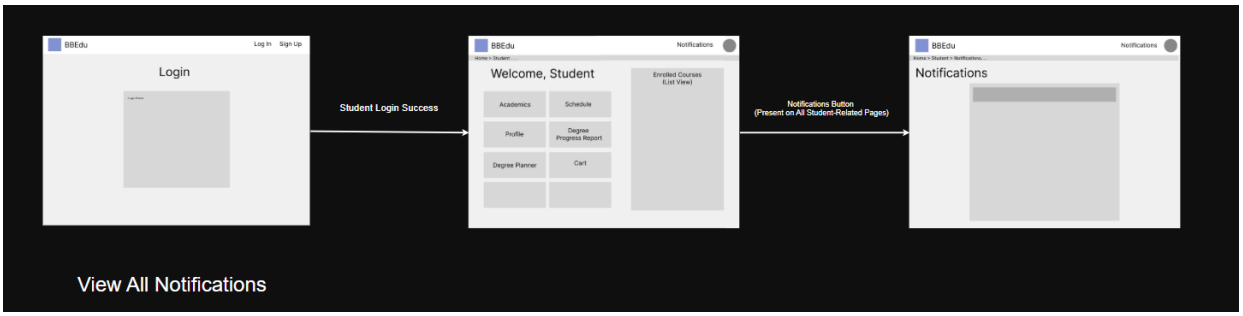
After authentication, the system redirects them based on role: students go to the Student Portal, while admins go to the Admin Dashboard.

2: Update Student Information (Student)



After logging in, the student goes to the Student Portal, opens Student Profile through the profile button or user icon, and can update their student or contact information from the profile page.

3. View All Notifications (Student)



After logging in, the student goes to the Student Portal and can access the Notifications page from any student-related page using the Notifications button.

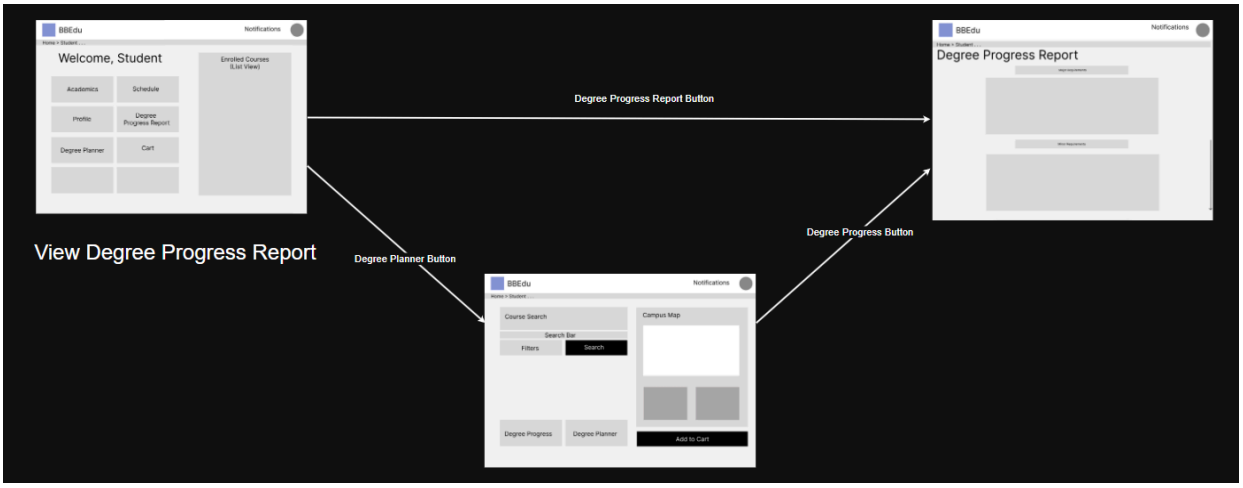
4. Viewing/Adding Courses & Campus Map (Student)

Students can open Academics to search for courses and view every campus location. When a course section is selected, the related building is highlighted on the campus map. The page also displays building details such as the building name and type, helping students understand where their class is located before adding it to the cart. Courses added to the cart will reflect in the Cart page. It also appears in the Degree Planner, where students can see how those courses fit into their planned academic path.

5. Enrolling in Classes (Student)

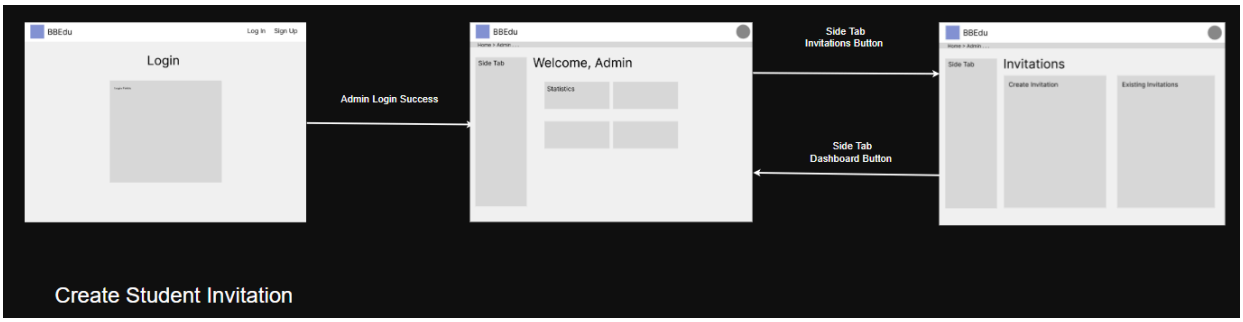
As mentioned previously, students can use Academics to search for course sections, view campus map details, and add selected classes to the cart. The Cart lets students review chosen classes, check requirements or conflicts, and proceed to enroll. After enrollment, the enrolled classes appear back on the Student Portal in the right-hand panel.

6. View Degree Progress Report (Student)



Students can open the Degree Progress Report from the Student Portal or Academics page. The report shows how completed, enrolled, or planned courses count toward major and other requirements.

7. Create Student Invitation (Admin)



Admins can access the Invitations page from the Admin Dashboard side tab after logging in. From there, they can create new student invitations and review existing invitations.

High Level System Design

Database Architecture

Functional Requirements

Entity Name/table_name (Type)

1. **User/app_user (Strong)**
    a. A User shall have exactly one unique user ID.
    b. A User shall reference exactly one school ID.
    c. A User shall have exactly one institutional email address.
    d. A User shall have exactly one password hash.
    e. A User shall have exactly one first name.
    f. A User shall have exactly one last name.
    g. A User shall have exactly one role (student or admin)
    h. A User shall have exactly one created timestamp.
    i. A User may have zero or one last login timestamp.
    j. A User may receive zero or many Notifications.
2. **AccountInvitation/account_invitation (Strong)**
    a. An Account Invitation shall have exactly one unique invitation ID.
    b. An Account Invitation shall have exactly one unique external ID.
    c. An Account Invitation shall have exactly one user role (student or admin).
    d. An Account Invitation shall have exactly one first name.
    e. An Account Invitation shall have exactly one last name.
    f. An Account Invitation may have zero or one phone number.
    g. An Account Invitation may have zero or one zip code.
    h. An Account Invitation may have zero or one institutional email address.
    i. An Account Invitation may reference zero or one Admin who created the invitation.
    j. An Account Invitation may have zero or one claimed timestamp.
    k. An Account Invitation shall have exactly one created timestamp.
    l. An Account Invitation external ID shall be unique.
    m. An Account Invitation institutional email shall be unique when provided.
3. **Student/student (Strong)**
    a. A Student shall reference exactly one user ID.
    b. A Student shall have exactly one academic level (freshman, sophomore, junior, senior, graduate).
    c. A Student shall have exactly one student type (first-time, continuing, transfer, international)
    d. A Student shall reference exactly one DegreeProgram.
    e. A student shall have exactly one total credits.
    f. A Student shall have exactly one expected graduation term.
    g. A Student shall have exactly one city.
    h. A Student shall have exactly one street.

i. A Student shall have exactly one state.
j. A Student shall have exactly one zipcode.
k. A Student may have zero or one phone number.
l. A Student shall have exactly one walking-speed value. (in meters/second)
m. A Student may have zero or many Enrollments.
n. A Student may have zero or many PlannedCourses.
o. A Student may have zero or many items in ClassCart.
p. A Student may have zero or one BillingProfile.
q. A Student may have zero or many StudentCharges.
r. A Student may have zero or many FinancialAid awards.
s. A Student may have zero or many StudentHolds.
t. A Student may be associated with zero or many classes through PlannedCourse.

4. **Admin/admin (Strong)**
   a. An Admin shall reference exactly one user ID.
   b. An Admin shall have exactly one unique institutional employee ID.
5. **InterfaceSetting/interface_setting (Strong)**
   a. An InterfaceSetting shall have exactly one unique setting ID.
   b. An InterfaceSetting shall reference exactly one admin ID.
   c. An InterfaceSetting shall have exactly one setting name. (header, sidebar…)
   d. An InterfaceSetting shall have exactly one setting value. (color, number, text)
   e. An InterfaceSetting shall have exactly one last updated timestamp.
6. **School/school (Strong)**
   a. A School shall have exactly one unique school ID.
   b. A School shall have exactly one name.
   c. A School may have zero or many Users.
   d. A School may have zero or many Departments.
   e. A School may have zero or many Buildings.
   f. A School may have zero or many Semesters.
   g. A School may have zero or many GEAreas.
7. **Department/department (Strong)**
   a. A Department shall have exactly one unique department ID.
   b. A Department shall reference exactly one school ID.
   c. A Department shall have exactly one department name.
   d. A Department shall reference exactly one building ID.
   e. A Department shall have exactly one office email address.
   f. A Department shall have exactly one office phone number.
   g. A Department may have zero or many Majors.
   h. A Department may have zero or many Courses.
   i. A Department (school ID, department name) pair shall be unique.
8. **Major/major (Strong)**
   a. A Major shall have exactly one unique major ID.
   b. A Major shall reference exactly one Department.
   c. A Major shall have exactly one name.
   d. A Major may have zero or many DegreePrograms.
9. **DegreeProgram/degree_program (Strong)**
   a. A DegreeProgram shall have exactly one unique degree program ID.

b. A DegreeProgram shall reference exactly one major ID.
c. A DegreeProgram shall have exactly one degree type (BA, BS, BFA, MA, MS, or PHD)
d. A DegreeProgram shall have exactly one catalog year.
e. A DegreeProgram shall have one required major units value.
f. A DegreeProgram shall have one required general education units value.
g. A DegreeProgram (major ID, degree type, catalog year) triple shall be unique.

10. Semester/semester (Strong)

a. A Semester shall have exactly one unique semester ID.
b. A Semester shall reference exactly one school ID.
c. A Semester shall have exactly one term year.
d. A Semester shall have exactly one term type (Fall, Spring, Summer, Winter)
e. A Semester shall have exactly one start date.
f. A Semester shall have exactly one end date.
g. A Semester shall have exactly one add/drop deadline.
h. A Semester shall have exactly one withdrawal deadline.
i. A Semester (school ID, term year, term type) combination shall be unique.

11. Course/course (Strong)

a. A Course shall have exactly one unique course ID.
b. A Course shall reference exactly one department ID.
c. A Course shall have exactly one subject code.
d. A Course shall have exactly one course number.
e. A Course shall have exactly one course title.
f. A Course shall have exactly one course description.
g. A Course shall have exactly one course units value.
h. A Course shall have exactly one course level (lower division, upper division, or graduate)
i. A Course shall have exactly one repeatable status.
j. A Course shall have exactly one section type (lecture, lab).
k. A Course may be associated with zero or more DegreeRequirement through RequiredCourse.
l. A Course may be associated with zero or more courses through CoursePrerequisite.
m. A Course may be associated with zero or many GEAreas through CourseGEArea.

12. ClassSection/class_section (Strong)

a. A ClassSection shall have exactly one unique class section ID.
b. A ClassSection shall reference exactly one course ID.
c. A ClassSection shall reference exactly one semester ID.
d. A ClassSection shall reference zero or one classroom ID.
e. A ClassSection shall reference exactly one instructor.
f. A ClassSection shall have exactly one section number.
g. A ClassSection shall have exactly one start date.
h. A ClassSection shall have exactly one end date.
i. A ClassSection shall have exactly one meeting start time.
j. A ClassSection shall have exactly one meeting end time.
k. A ClassSection may have zero or more MeetingDays.

l. A ClassSection shall have exactly one modality (in-person, online, hybrid)
m. A ClassSection shall have exactly one meeting type (synchronous/asynchronous)
n. A ClassSection shall have exactly one capacity.
o. A ClassSection shall have exactly one enrolled count.
p. A ClassSection shall have exactly one waitlist capacity.
q. A ClassSection shall have exactly one waitlist count.
r. A ClassSection shall have exactly one status (open, closed, waitlist, cancelled)

13. **MeetingDay/meeting_day (Weak)**
    a. A MeetingDay shall reference exactly one ClassSection ID.
    b. A MeetingDay shall have exactly one day of the week.
    c. A MeetingDay (Class section ID, day of week) pair shall be unique.
14. **Instructor/instructor (Strong)**
    a. An Instructor shall have exactly one unique instructor ID.
    b. An Instructor shall have exactly one first name.
    c. An Instructor shall have exactly one last name.
    d. An Instructor shall have exactly one institutional email.
15. **Building/building (Strong)**
    a. A Building shall have exactly one unique building ID.
    b. A Building shall reference exactly one school ID.
    c. A Building shall have exactly one name.
    d. A Building may have one or many BuildingTypes.
    e. A Building shall have zero or more Classrooms.
    f. A Building shall have exactly one unique map element ID. (for frontend)
    g. A Building may be associated with one or more buildings in BuildingDistance.
    h. A Building (school ID, building name) pair shall be unique.
    i. A Building (school ID, map element ID) pair shall be unique.
16. **BuildingType/building_type (Weak)**
    a. A BuildingType shall reference exactly one building ID.
    b. A BuildingType shall have exactly one building type. (academic, gym, library, student center, dining, administration, parking)
    c. A BuildingType (building ID, building type) pair shall be unique.
17. **BuildingDistance/building_distance (Associative)**
    a. A BuildingDistance shall reference exactly one origin building ID.
    b. A BuildingDistance shall reference exactly one destination building ID.
    c. A BuildingDistance shall have exactly one distance value. (in meters)
    d. A BuildingDistance (origin building ID, destination building ID) pair shall be unique.
18. **Classroom/classroom (Strong)**
    a. A Classroom shall have exactly one unique classroom ID.
    b. A Classroom shall reference exactly one building ID.
    c. A Classroom shall have exactly one room number.
19. **EnrollmentWindow/enrollment_window (Strong)**
    a. An EnrollmentWindow shall have exactly one unique window ID.
    b. An EnrollmentWindow shall reference exactly one semester ID.
    c. An EnrollmentWindow shall have exactly one student type.
    d. An EnrollmentWindow shall have exactly one academic level.

e. An EnrollmentWindow shall have exactly one enrollment start timestamp.
f. An EnrollmentWindow shall have exactly one enrollment end timestamp.
g. An EnrollmentWindow (semester_id, student_type, academic_level) triple shall be unique.

20. **Enrollment/enrollment (Associative)**
    a. An Enrollment shall reference exactly one student ID.
    b. An Enrollment shall reference exactly one class section ID.
    c. An Enrollment shall have exactly one enrollment date.
    d. An Enrollment shall have exactly one enrollment status (enrolled, dropped, withdrawn, completed)
    e. An Enrollment may have zero or one grade value (A-F, P/NP)
    f. An Enrollment (student ID, class section ID) pair shall be unique.
21. **StudentHold/student_hold (Strong)**
    a. A StudentHold shall have exactly one unique hold ID.
    b. A StudentHold shall reference exactly one student ID.
    c. A StudentHold shall have exactly one hold type.
    d. A StudentHold shall have exactly one hold status.
    e. A StudentHold shall have exactly one placed date.
    f. A StudentHold may have zero or one resolved date.
    g. A StudentHold shall have exactly one enrollment-blocking indicator.
22. **PlannedCourse/planned_course (Associative)**
    a. A PlannedCourse shall reference exactly one student ID.
    b. A PlannedCourse shall reference exactly one course ID.
    c. A PlannedCourse shall reference exactly one semester ID.
    d. A PlannedCourse shall have exactly one added date.
    e. A PlannedCourse (student ID, course ID, semester ID) combination shall be unique.
23. **ClassCart/class_cart (Associative)**
    a. A ClassCart shall reference exactly one student ID.
    b. A ClassCart shall reference exactly one class section ID.
    c. A ClassCart shall have exactly one added date.
    d. A ClassCart(student ID, class section ID) pair shall be unique.
24. **CoursePrerequisite/course_prerequisite (Associative)**
    a. A CoursePrerequisite shall reference exactly one course ID.
    b. A CoursePrerequisite shall reference exactly one prerequisite course ID.
    c. A CoursePrerequisite may have zero or one minimum grade required.
    d. A CoursePrerequisite (course ID, prerequisite course ID) pair shall be unique.
25. **DegreeRequirement/degree_requirement (Strong)**
    a. A DegreeRequirement shall have exactly one unique degree requirement ID.
    b. A DegreeRequirement shall reference exactly one degree program ID.
    c. A DegreeRequirement may reference zero or one GE area ID.
    d. A DegreeRequirement shall have exactly one requirement name.
    e. A Degree requirement shall have exactly one requirement type.(major-core, major-elective, GE Area, university-requirement)
    f. A DegreeRequirement shall have exactly one required units value.
    g. A DegreeRequirement may have zero or one minimum grade required.
26. **CourseGEArea/course_gearea (Associative)**

a. A CourseGEArea shall reference exactly one course ID.
b. A CourseGEArea shall reference exactly one GE area ID.
c. A CourseGEArea (course ID, GE area ID) pair shall be unique.

27. **RequiredCourse/required_course (Associative)**
    a. A RequiredCourse shall reference exactly one requirement ID.
    b. A RequiredCourse shall reference exactly one course ID.
    c. A RequiredCourse (requirement_id, course_id) pair shall be unique.
28. **StudentCharge/student_charge (Strong)**
    a. A StudentCharge shall have exactly one unique charge ID.
    b. A StudentCharge shall reference exactly one student ID.
    c. A StudentCharge shall reference exactly one semester ID.
    d. A StudentCharge shall have exactly one charge type (tuition, insurance, parking, student pass, …etc)
    e. A StudentCharge shall have exactly one charge amount. (in USD)
    f. A StudentCharge shall have exactly one due date.
    g. A StudentCharge shall have exactly one charge type (tuition, fees, housing, meal plan, parking, insurance, student pass, or other).
    h. A StudentCharge shall have exactly one charge status (pending, paid, overdue, waived, refunded, or cancelled).
29. **FinancialAid/financial_aid (Strong)**
    a. A FinancialAid shall have exactly one unique aid ID.
    b. A FinancialAid shall reference exactly one student ID.
    c. A FinancialAid shall reference exactly one semester ID.
    d. A FinancialAid shall have exactly one aid type (grant, loan, scholarship, work-study).
    e. A FinancialAid shall have exactly one aid name.
    f. A FinancialAid shall have exactly one amount. (in USD)
    g. A FinancialAid shall have exactly one status (pending, accepted, declined, disbursed).
    h. A FinancialAid may have zero or one disbursement date.
    i. A FinancialAid may have zero or one accepted date.
30. **BillingProfile/billing_profile (Strong)**
    a. A BillingProfile shall have exactly one unique billing profile ID.
    b. A BillingProfile shall reference exactly unique one student ID.
    c. A BillingProfile shall have exactly one city.
    d. A BillingProfile shall have exactly one street.
    e. A BillingProfile shall have exactly one state.
    f. A BillingProfile shall have exactly one zipcode.
    g. A BillingProfile may have zero or more BillingTransactions.
31. **BillingTransaction/billing_transaction (Strong)**
    a. A BillingTransaction shall have exactly one unique transaction ID.
    b. A BillingTransaction shall reference exactly one billing profile ID.
    c. A BillingTransaction shall have exactly one transaction type (payment, refund, or financial aid)
    d. A Billing Transaction shall have exactly one transaction amount (in USD)
    e. A BillingTransaction shall have exactly one transaction timestamp.

f. A Billing Transaction shall have exactly one status (pending , completed, or failed)

32. Notification/notification (Strong)

a. A Notification shall have exactly one unique notification ID.
b. A Notification shall reference exactly one user ID.
c. A Notification shall exactly one notification type (enrollment, deadline, schedule-conflict, walking-time-conflict, payment, general)
d. A Notification shall have exactly one title.
e. A Notification shall have exactly one message.
f. A Notification shall have exactly one created timestamp.

Entity Descriptions

Type	Entities	Description
User and Access Management	School, User, Student, Admin, InterfaceSetting, AccountVerification	Stores schools, user accounts, student and administrator information, and interface settings managed by administrators. Verifies account information to sign-up.
Academic Organization	Department, Major, DegreeProgram, GEArea	Stores the academic structure of each school, including departments, majors, degree programs, catalog years, and general education areas.
Course Catalog	Course, CoursePrerequisite, CourseGEArea	Stores course information, prerequisite relationships, and the general education areas associated with each course.
Degree Requirements	DegreeRequirement, RequiredCourse	Stores graduation requirement categories and the courses that may satisfy each requirement.
Class Scheduling	Semester, ClassSection, MeetingDay, Instructor	Stores academic terms and specific course offerings, including instructors, meeting dates, times, days, modality, capacity, and enrollment availability.
Enrollment and Course Planning	EnrollmentWindow, Enrollment, StudentHold, PlannedCourse, ClassCart	Stores enrollment periods, student enrollment records, enrollment-blocking holds, long-term course plans, and class sections selected for registration.
Campus Navigation	Building, BuildingType, Classroom, BuildingDistance	Stores campus buildings, building categories, classrooms, and distances between buildings.
Student Financial Information	StudentCharge, FinancialAid, BillingProfile, BillingTransaction	Stores student charges, financial aid awards, billing addresses, and payment, refund, or financial-aid

		transaction records.
Notifications	Notification	Stores system-generated messages about enrollment, deadlines, schedule conflicts, walking-time conflicts, payments, and other important updates.

Several key features of our application – Calendar, Degree Planner, and Campus Map – are not stored as database entities. They are user-interface features generated from the entities above.

The Calendar uses class-section, meeting-day, enrollment, and cart data; the Degree Planner uses degree-program, degree-requirement, enrollment, and planned-course data; and the Campus Map uses building, classroom, map-element, and building-distance data.

Normalization Check

First Normal Form (1NF)

Our schema satisfies 1NF because each attribute stores a single atomic value instead of a list or repeating group. For example, first_name and last_name are stored separately instead of storing a full name in one field. Student location information is also separated into fields such as street, city, state, and zip_code, so each part of the address is individually stored. Course information is separated into fields such as subject_code, course_number, course_title, course_description, and course_units. Multi-valued facts are also moved into separate tables, such as meeting_day for multiple class meeting days, building_type for multiple building categories, course_gearea for multiple GE areas, and course_prerequisite for course prerequisites. This prevents columns from containing comma-separated or repeated values.

Second Normal Form (2NF)

Our schema satisfies 2NF because every non-key attribute depends on the full primary key of its table. For tables with single primary keys, such as app_user, student, course, class_section, building, and instructor, the attributes depend on that entity's ID. For example, in app_user, attributes such as institutional_email, password_hash, first_name, last_name, and user_role depend on user_id. In course, attributes such as course_title, course_description, course_units, course_level, and section_type depend on course_id. For composite-key tables like enrollment, class_cart, planned_course, course_gearea, and required_course, the non-key attributes describe the full relationship, not just one part of the key.

Third Normal Form (3NF)

Our schema satisfies 3NF because non-key attributes depend only on the key and not on other non-key attributes. For example, user login and identity information is stored in app_user, while student-specific information such as academic level, student type, total credits, expected graduation term, phone number, and walking speed is stored in student.

Course details are stored in course, while section-specific details such as meeting time, capacity, enrolled count, waitlist count, modality, classroom, instructor, and semester are stored in class_section. Instructor names and emails are stored in ‘instructor’, building names and map element IDs are stored in ‘building’, and room numbers are stored in ‘classroom’. This keeps each table focused on one subject and avoids storing unrelated details in the wrong table.

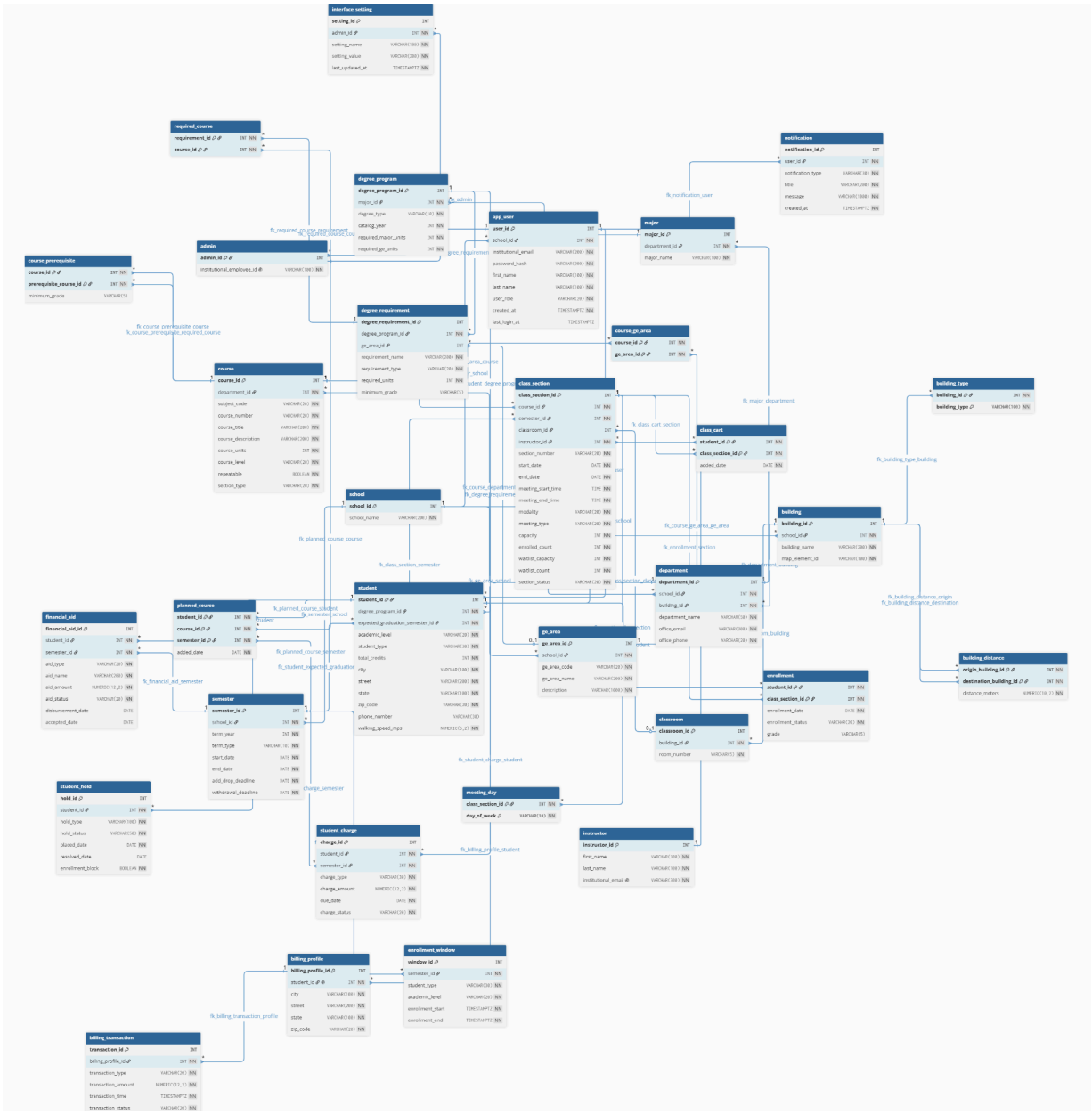
Boyce-Codd Normal Form (BCNF)

Our schema satisfies BCNF because the main determinants in the schema are candidate keys or are enforced through unique constraints. For example, user_id determines the user’s email, name, role, password hash, created timestamp, and last login timestamp. course_id determines course details, while the combination of department_id, subject_code, and course_number is unique for identifying a course within a department. class_section_id determines section details, and the combination of course_id, semester_id, and section_number is unique for identifying a specific section offering. Other unique combinations, such as building name within a school, semester term within a school, and degree program within a major/catalog year, help prevent conflicting duplicate records.

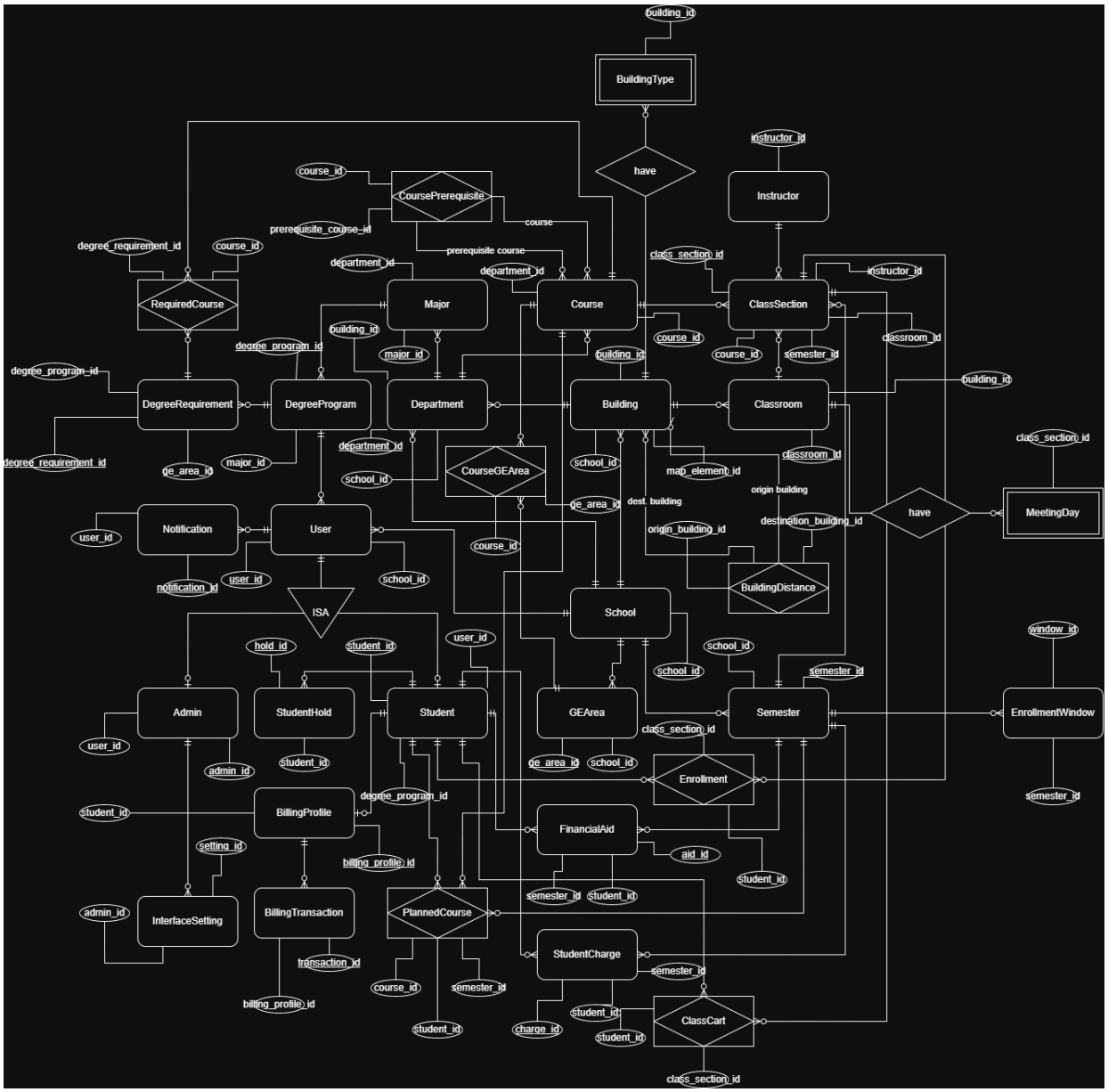
Fourth Normal Form (4NF)

Our schema satisfies 4NF because independent multi-valued relationships are separated into their own tables instead of being mixed into one table. A class section can meet on multiple days, so those days are stored in meeting_day. A building can have multiple types, so those are stored in building_type. A course can satisfy multiple GE areas, so course_gearea handles that relationship. A requirement can be satisfied by multiple courses, so required_course handles that. A student can have multiple enrollments, planned courses, cart items, charges, holds, and financial aid awards, so those are separated into their own tables. This avoids storing unrelated repeated facts together and reduces duplication.

ERD (Entity-Relationship Diagram)



EER (Enhanced Entity-Relationship) Diagram



Backend Architecture

The backend uses a modular monolithic architecture built with Node.js and Express.js. It provides REST APIs for the React frontend and connects to an Amazon RDS PostgreSQL database through the pg connection pool.

Requests follow a route-controller-database structure. Routes define API endpoints and apply authentication middleware. Controllers validate requests, execute business logic, and return JSON responses. The database layer uses parameterized SQL queries and transactions to maintain security and data consistency.

React Frontend → Express Route → Authentication Middleware → Controller → PostgreSQL → JSON Response

Backend Modules

The backend is divided into modules based on application features:

- Authentication: Handles account activation, login, logout, session validation, and role-based access.
- Student Profile: Allows students to view and update profile information.
- Course Catalog:Provides courses, class sections, semesters, departments, and building information.
- Planned Schedule and Cart: Manages selected class sections and checks schedule conflicts.
- Student Planning: Provides the calendar, degree progress, and walking-time warnings.
- Admin Management: Allows administrators to manage accounts, courses, sections, buildings, classrooms, and degree requirements.
- Health Check:Verifies that the backend server and database are available.

Security

The backend applies security controls through password protection, token-based authentication, role-based authorization, request validation, safe database access, transaction management, environment-based configuration, and standardized error responses.

- Password Hashing and Login Verification
    - User passwords are never stored as plaintext. When an account is created, the backend generates a unique random salt and hashes the password using Node.js's `scrypt` password-based key derivation function. The salt and resulting hash are stored together in the user's `password_hash` field.
    - During login, the backend hashes the submitted password using the stored salt and compares the result with the saved password hash. A timing-safe comparison function is used to reduce the risk of timing attacks. New account passwords must contain at least eight characters.

- Bearer Token Authentication
    - After a successful login, the backend generates an HMAC-SHA256 signed bearer token. The token contains the authenticated user's ID, assigned role, and expiration time. Tokens expire after eight hours.
    - Clients include the token in protected API requests using the following HTTP header:
        - Authorization: Bearer <token>
        - Before processing a protected request, the backend verifies the token's signature, expiration time, and revocation status. Invalid, expired, or revoked tokens are rejected.
        - When a user logs out, the current token is added to an in-memory revocation list and cannot be reused during its remaining lifetime.

- Authentication and Authorization Middleware
    - Protected API routes use the requireAuth middleware to verify the bearer token and identify the user making the request. Administrative routes use requireAdmin, which also checks that the authenticated user has the admin role.
    - This prevents students from accessing administrative APIs directly, even if they bypass the frontend. Admin-only operations include managing account invitations, courses, class sections, buildings, classrooms, degree requirements, and student academic information.

- Request Validation
    - Backend controllers validate incoming request data before performing business logic or database operations. Validation includes checking required fields, password length, supported user roles, record identifiers, and expected data types or values.
    - Invalid requests are rejected before they can modify application data. The backend also checks for conditions such as missing records, duplicate selections, schedule conflicts, invalid account invitations, and unauthorized administrative operations.
    - Additional validation can be added through reusable validation middleware as the API grows.

- Parameterized SQL Queries
    - Backend controllers use PostgreSQL parameterized queries. User-provided values are passed through placeholders such as "2" and "1" instead of being directly inserted into SQL strings.
    - This separates input data from SQL instructions and reduces the risk of SQL injection. Parameterized queries are used in authentication, student profile, course management, planned schedule, class cart, student planning, and administrative operations.
    - Database-level security controls and constraints are documented separately in the Database Architecture section.

- Transaction Management

    - Backend controllers use PostgreSQL transactions for operations that modify multiple related records. A transaction begins before the changes are made and is committed only after every required operation succeeds.
    - If validation fails or a database operation produces an error, the controller performs a rollback. This prevents partial updates and keeps related application operations consistent.
    - Transactions are currently used for operations including account activation, profile updates, planned schedule changes, class cart changes, and class section management.

- Environment-Based Configuration
    - Sensitive backend configuration is read from environment variables instead of being directly included in application logic.
    - The backend currently uses environment variables including:
    - AUTH_TOKEN_SECRE for signing authentication tokens
    - DATABASE_URL for the PostgreSQL connection
    - DATABASE_SSL for controlling the database SSL connection
    - PORT for configuring the backend server port
    - The production environment must provide a strong, randomly generated AUTH_TOKEN_SECRET. The development fallback secret must not be used in production.

- Error Status Codes and Responses
    - The backend returns HTTP status codes that distinguish different types of failures:
        - 400 Bad Request: for missing or invalid request data
        - 401 Unauthorized: for missing, invalid, expired, or revoked tokens
        - 403 Forbidden: for authenticated users without the required role
        - 404 Not Found: when the requested resource does not exist
        - 409 Conflict: for duplicate or conflicting operations
        - 500 Internal Server Error: for unexpected server failures
    - Error responses return JSON messages without exposing database credentials, password hashes, authentication secrets, or internal stack traces to the client.

- CORS and API Protection
    - The Express server currently enables CORS so that the React frontend can communicate with the backend API. The current prototype accepts requests from unrestricted origins.
    - For production deployment, CORS should be configured to accept requests only from the approved frontend domain. The backend should also add rate limiting to authentication endpoints to reduce brute-force login attempts.
    - Additional planned API protections include request-body size limits, centralized security logging, automated authentication tests, and dependency vulnerability scanning.

Validation and Correctness

- Data Validation
    - Controllers validate request data before performing an operation. They check required fields, record identifiers, expected data types, and supported values. Invalid input is rejected before it can create or modify application records.
    - Backend also confirms that referenced records exist before continuing. For example, a requested course, class section, student, or account invitation must be found before the related operation can be completed.

- Business Rule Enforcement
    - Before making changes to the database, the backend checks whether the requested action is valid for the current user and application state. For example, it prevents students from adding the same class to their Planned Schedule or Class Cart more than once and checks for time conflicts between selected classes
    - The backend also verifies that a Class Section is available before adding it to a schedule or cart. During account activation, it confirms that the invitation exists and has not already been claimed. Similar checks are performed before profile updates, administrative changes, and operations involving related records

- Transaction Management
    - The backend uses PostgreSQL transactions when one request needs to update several related records. Changes are saved only after the entire operation completes successfully. If any step fails, the backend rolls back the transaction so the database is not left with incomplete or partially updated data
    - The backend uses PostgreSQL transactions when one request needs to update several related records. Changes are saved only after the entire operation completes successfully. If any step fails, the backend rolls back the transaction so the database is not left with incomplete or partially updated data.
    - Transactions are used in account activation, student profile updates, Planned Schedule and Class Cart changes, and the management of Class Sections and their Meeting Days.

- Concurrent Request Handling
    - During account activation, the backend temporarily locks the selected invitation so it cannot be used by another request at the same time. The invitation is marked as claimed only after the account has been created successfully. If the process fails, the changes are rolled back and the invitation remains available.

- Error Responses
    - Controllers use try/catch blocks to handle invalid operations, unavailable records, conflicts, and unexpected database failures. When an error occurs

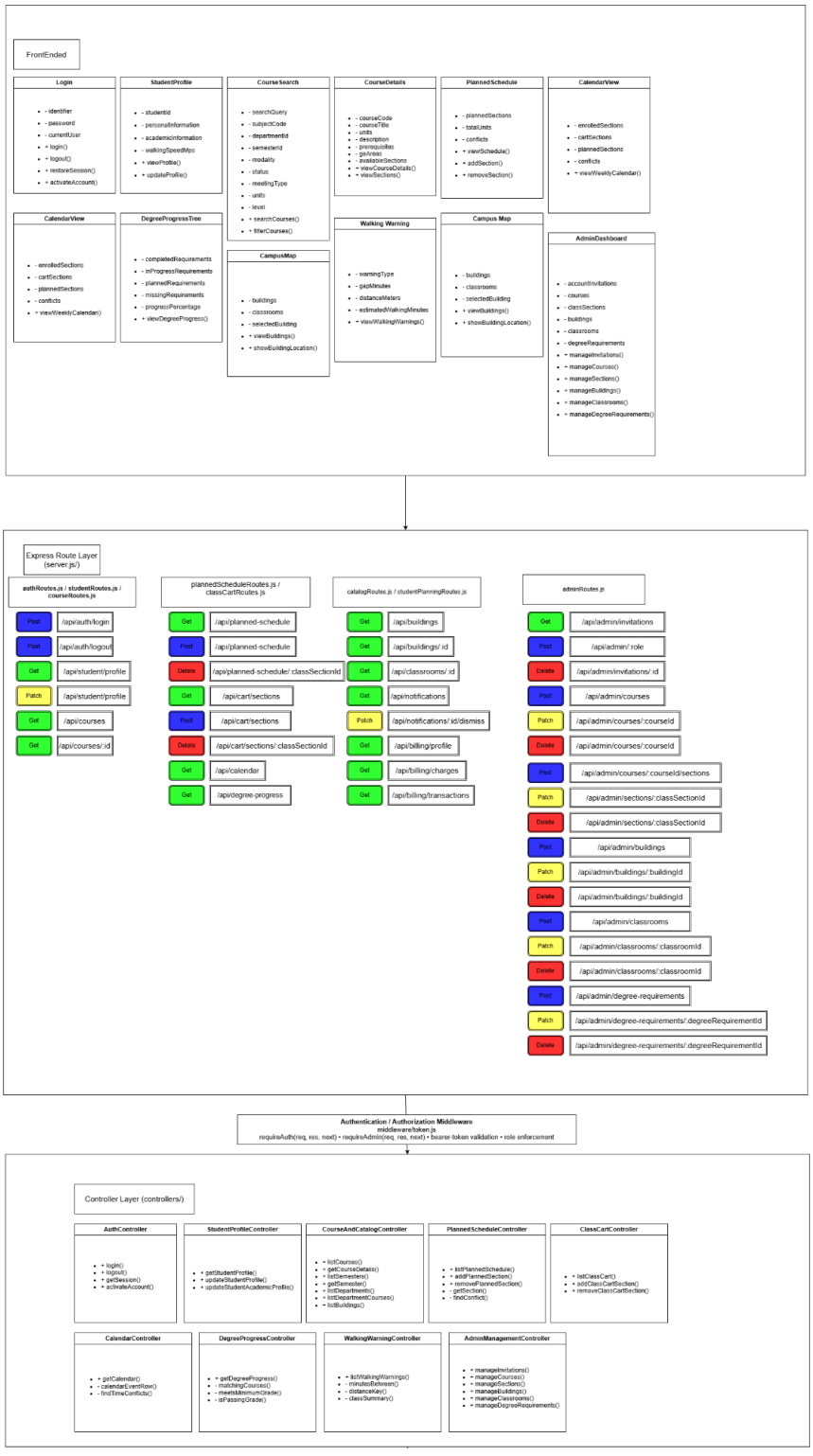
during a transaction, the backend rolls back the operation before returning a response.

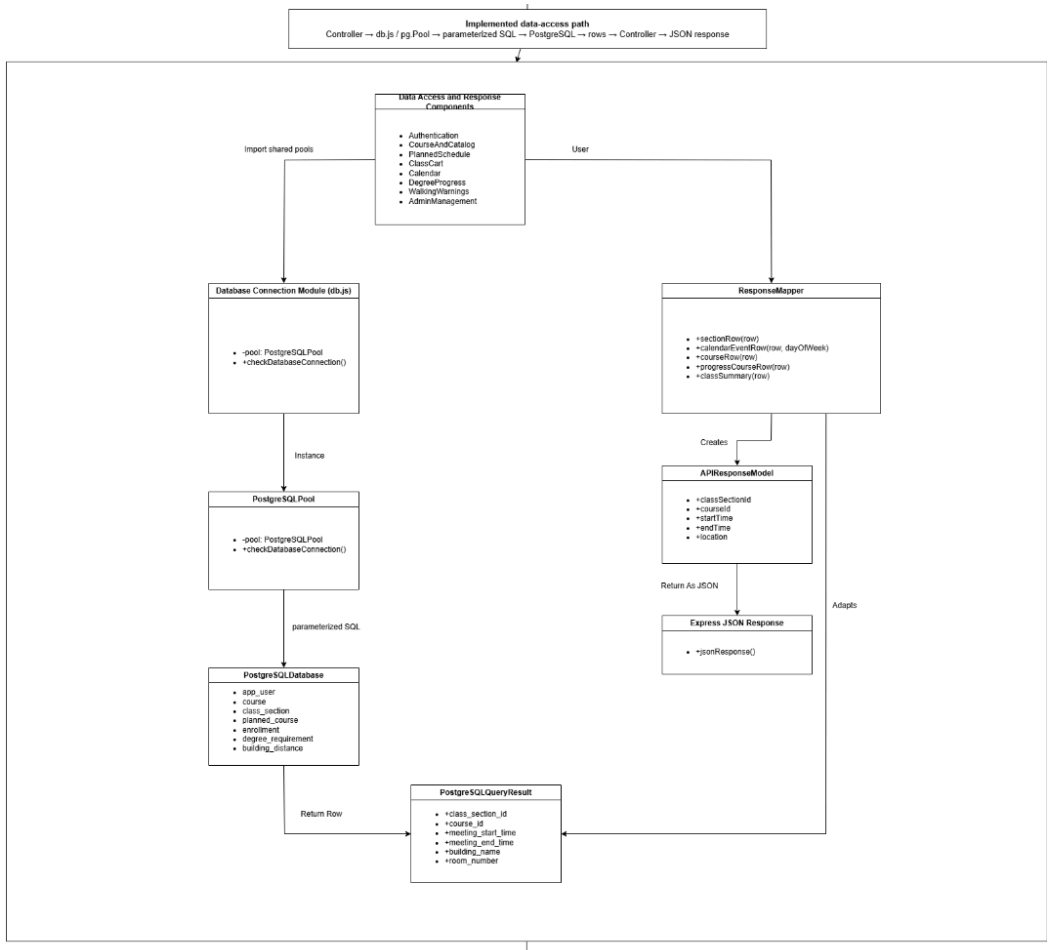
- Connection Management
    - The backend uses a PostgreSQL connection pool to reuse database connections. Controllers that acquire a dedicated database client release it after the operation succeeds or fails. This prevents connection leaks and supports reliable handling of concurrent requests.

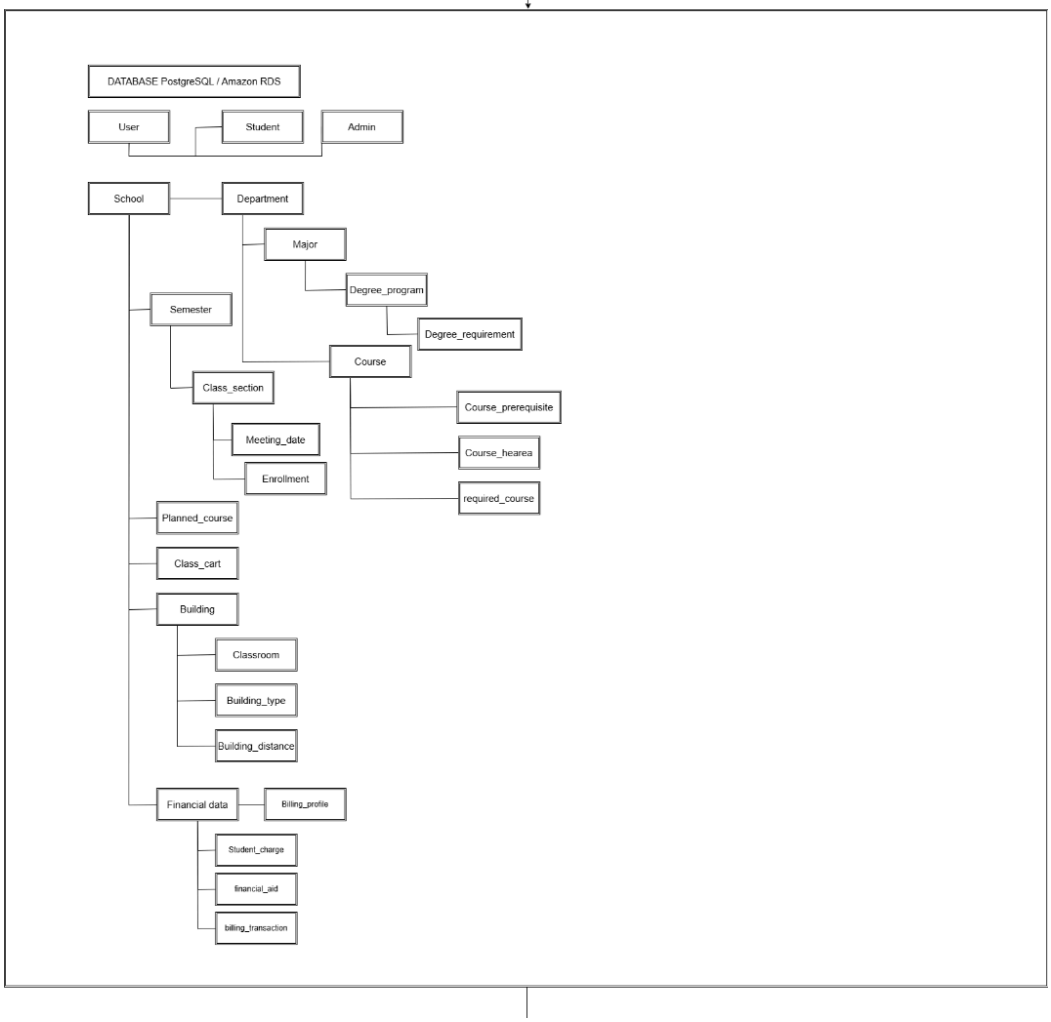
Scalability

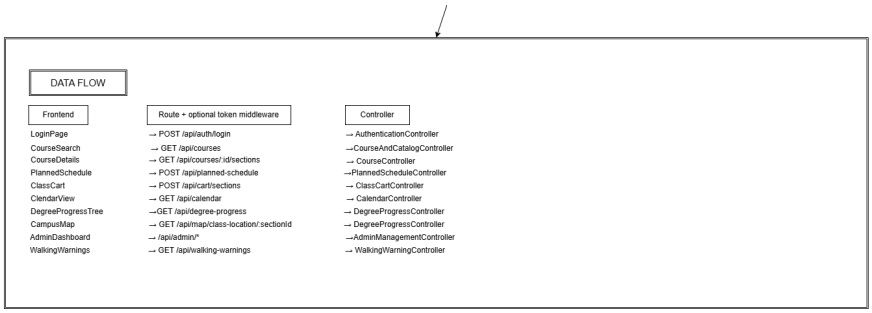
- The backend uses a PostgreSQL connection pool instead of creating a new database connection for every request. This allows connections to be reused and helps the server handle multiple requests more efficiently. In the deployed environment, Nginx receives incoming traffic, the Node.js/Express backend runs on AWS EC2, and application data is stored in Amazon RDS PostgreSQL.
- If traffic increases, we can run multiple backend instances behind Nginx or an AWS load balancer. One issue that must be addressed first is token revocation. Revoked tokens are currently stored in server memory, so multiple backend instances would not share the same revocation information. Moving this data to Redis or PostgreSQL would allow all instances to use the same revocation list.
- Other practical improvements include adding pagination to large API responses, caching frequently requested course information, monitoring database connections, and adding indexes for commonly searched fields. These improvements can be introduced gradually as the amount of data and number of users increase. For the current project scope, the modular monolithic architecture provides enough scalability without adding unnecessary deployment complexity.

UML


FrontEnded
Login
- identifier
- password
- currentUser
+ login()
+ logout()
+ restoreSession()
+ activateAccount()
StudentProfile
- studentId
- personalInformation
- academicInformation
- walkingSpeedMps
+ viewProfile()
+ updateProfile()
CourseSearch
- searchQuery
- subjectCode
- departmentId
- semesterId
- modality
- status
- meetingType
- units
- level
+ searchCourses()
+ filterCourses()
CourseDetails
- courseCode
- courseTitle
- units
- description
- prerequisites
- gpaAreas
- availableSections
+ viewCourseDetails()
+ viewSections()
PlannedSchedule
- plannedSections
- totalUnits
- conflicts
+ viewSchedule()
+ addSection()
+ removeSection()
CalendarView
- enrolledSections
- cartSections
- plannedSections
- conflicts
+ viewWeeklyCalendar()
CalendarView
- enrolledSections
- cartSections
- plannedSections
- conflicts
+ viewWeeklyCalendar()
DegreeProgressTree
- completedRequirements
- inProgressRequirements
- plannedRequirements
- missingRequirements
- progressPercentage
+ viewDegreeProgress()
CampusMap
- buildings
- classrooms
- selectedBuilding
+ viewBuildings()
+ showBuildingLocation()
Walking Warning
- warningType
- gapMinutes
- distanceMeters
- estimatedWalkingMinutes
+ viewWalkingWarnings()
Campus Map
- buildings
- classrooms
- selectedBuilding
+ viewBuildings()
+ showBuildingLocation()
AdminDashboard
- accountInvitations
- courses
- classSections
- buildings
- classrooms
- degreeRequirements
+ manageInvitations()
+ manageCourses()
+ manageSections()
+ manageBuildings()
+ manageClassrooms()
+ manageDegreeRequirements()
Express Route Layer (server.js/)
authRoutes.js / studentRoutes.js / courseRoutes.js
Post /api/auth/login
Post /api/auth/logout
Get /api/student/profile
Patch /api/student/profile
Get /api/courses
Get /api/courses/:id
plannedScheduleRoutes.js / classCartRoutes.js
Get /api/planned-schedule
Post /api/planned-schedule
Delete /api/planned-schedule/:classSectionId
Get /api/cart/sections
Post /api/cart/sections
Delete /api/cart/sections/:classSectionId
Get /api/calendar
Get /api/degree-progress
catalogRoutes.js / studentPlanningRoutes.js
Get /api/buildings
Get /api/buildings/:id
Get /api/classrooms/:id
Get /api/notifications
Patch /api/notifications/:id/dismiss
Get /api/billing/profile
Get /api/billing/charges
Get /api/billing/transactions
adminRoutes.js
Get /api/admin/invitations
Post /api/admin/:role
Delete /api/admin/invitations/:id
Post /api/admin/courses
Patch /api/admin/courses/:courseId
Delete /api/admin/courses/:courseId
Post /api/admin/courses/:courseId/sections
Patch /api/admin/sections/:classSectionId
Delete /api/admin/sections/:classSectionId
Post /api/admin/buildings
Patch /api/admin/buildings/:buildingId
Delete /api/admin/buildings/:buildingId
Post /api/admin/classrooms
Patch /api/admin/classrooms/:classroomId
Delete /api/admin/classrooms/:classroomId
Post /api/admin/degree-requirements
Patch /api/admin/degree-requirements/:degreeRequirementId
Delete /api/admin/degree-requirements/:degreeRequirementId
Authentication / Authorization Middleware
middleware/token.js
requireAuth(req, res, next) • requireAdmin(req, res, next) • bearer-token validation • role enforcement
Controller Layer (controllers/)
AuthController
+ login()
+ logout()
+ getSession()
+ activateAccount()
StudentProfileController
+ getStudentProfile()
+ updateStudentProfile()
+ updateStudentAcademicProfile()
CourseAndCatalogController
+ listCourses()
+ getCourseDetails()
+ listSemesters()
+ getSemester()
+ listDepartments()
+ listDepartmentCourses()
+ listBuildings()
PlannedScheduleController
+ listPlannedSchedule()
+ addPlannedSection()
+ removePlannedSection()
- getSection()
- findConflict()
ClassCartController
+ listClassCart()
+ addClassCartSection()
+ removeClassCartSection()
CalendarController
+ getCalendar()
- calendarEventFlow()
- findTimeConflicts()
DegreeProgressController
+ getDegreeProgress()
- matchingCourses()
- meetsMinimumGrade()
- isPassingGrade()
WalkingWarningController
+ listWalkingWarnings()
- minutesBetween()
- distanceKey()
- classSummary()
AdminManagementController
+ manageInvitations()
+ manageCourses()
+ manageSections()
+ manageBuildings()
+ manageClassrooms()
+ manageDegreeRequirements()


Implemented data-access path
Controller → db.js / pg.Pool → parameterized SQL → PostgreSQL → rows → Controller → JSON response
Data Access and Response Components
Authentication
CourseAndCatalog
PlannedSchedule
ClassCart
Calendar
DegreeProgress
WaitingWarnings
AdminManagement
Import shared pools
User
Database Connection Module (db.js)
-pool: PostgreSQLPool
+checkDatabaseConnection()
ResponseMapper
+sectionRow(row)
+calendarEventRow(row, dayOfWeek)
+courseRow(row)
+progressCourseRow(row)
+classSummary(row)
Instance
Creates
APIResponseModel
+classSectionId
+courseId
+startTime
+endTime
+location
PostgreSQLPool
-pool: PostgreSQLPool
+checkDatabaseConnection()
Return As JSON
Adapts
Express JSON Response
+jsonResponse()
parameterized SQL
PostgreSQLDatabase
app_user
course
class_section
planned_course
enrollment
degree_requirement
building_distance
Return Row
PostgreSQLQueryResult
+class_section_id
+course_id
+meeting_start_time
+meeting_end_time
+building_name
+room_number


DATABASE PostgreSQL / Amazon RDS
User
Student
Admin
School
Department
Major
Degree_program
Degree_requirement
Semester
Course
Class_section
Course_prerequisite
Meeting_date
Course_hearea
Enrollment
required_course
Planned_course
Class_cart
Building
Classroom
Building_type
Building_distance
Financial data
Billing_profile
Student_charge
financial_aid
billing_transaction


DATA FLOW
Frontend
Route + optional token middleware
Controller
LoginPage → POST /api/auth/login → AuthenticationController
CourseSearch → GET /api/courses →CourseAndCatalogController
CourseDetails → GET /api/courses/ id/sections → CourseController
PlannedSchedule → POST /api/planned-schedule →PlannedScheduleController
ClassCart → POST /api/cart/sections → ClassCartController
ClendarView → GET /api/calendar → CalendarController
DegreeProgressTree →GET /api/degree-progress → DegreeProgressController
CampusMap → GET /api/map/class-location/:sectionId → DegreeProgressController
AdminDashboard → /api/admin/* →AdminManagementController
WalkingWarnings → GET /api/walking-warnings → WalkingWarningController

Project Management

For Milestone 3, the team will manage the project by focusing on three main checkpoints: technical documentation, horizontal prototype implementation, and software/code review. Since Milestone 3 requires both documentation and working software, the team will divide responsibilities based on each member's role while keeping major scope and priority decisions as a team discussion.

The first step for Milestone 3 is to finish the required revisions from Milestone 2 Version 2 and make sure the project documentation is consistent with the current implementation. After that, the team will finalize the Priority 1 and Priority 2 functional requirements. Priority 3 requirements will not be implemented during this course, so the team will focus on realistic features that can be completed and tested with good quality. This helps prevent the team from over-scoping the project.

The team will use GitHub as the main code collaboration platform. All members are expected to pull the latest default branch before starting work, create or use assigned branches when needed, write meaningful commit messages, and avoid committing sensitive files such as `.env`. Final graded work must be merged into the default branch before submission. The team will also use shared documentation tools such as Google Docs, Notion, or Markdown to organize Milestone 3 documentation, task ownership, deadlines, and progress tracking.

For the technical documentation checkpoint, Oscar Wang will coordinate document formatting and wireframe integration as the Technical Writer, while Wen-Chien Yen will review the final structure before submission. Sean Wang will update the database architecture, revised ERD, EER diagram, schema, and seed data. Banghao Yuan will update the backend architecture, API design, and implementation details. Kashif Zada will help review security practices, validation, and known issues. Enoch Lin will help verify GitHub organization, branch management, and software review readiness.

For the horizontal prototype checkpoint, the team will focus on breadth of user interface functionality. The prototype should allow users to navigate between the major sections defined in the wireframes, including signup, login, course search, and other planned Student Center pages. The team will prioritize clear navigation, consistent terminology, readable layouts, and useful feedback messages. The backend and database must continue supporting the main Priority 1 features, especially signup, login, course search, and database-backed results.

For the software review checkpoint, the team will check repository structure, coding standards, framework configuration, database organization, and security practices. This includes making sure code is readable, names are consistent, comments are useful, credentials are handled securely, and the database schema matches the documented data definitions. The team will also verify that the deployed prototype runs correctly on the cloud instance and that the README matches the actual deployed application.

Wen-Chien Yen, as Team Lead, will be responsible for coordinating progress, checking that required sections are completed, verifying deployment, reviewing final README and credentials instructions, and submitting required emails to the instructor and TA. Before each

checkpoint submission, the team will complete integration testing to confirm that the deployed prototype works from the public URL and that the frontend, backend, and database are connected correctly.

The overall Milestone 3 workflow is:

1. Complete Milestone 2 Version 2 revisions.
2. Finalize Priority 1 and Priority 2 scope.
3. Update M3v1 technical documentation.
4. Update UI/UX wireframes and Figma link.
5. Update database architecture, ERD, and EER diagram.
6. Update backend architecture and API documentation.
7. Build and test the horizontal prototype.
8. Verify cloud deployment and database connection.
9. Review GitHub organization, code quality, and security practices.
10. Prepare final submission materials and known issues.

Team Member	Milestone 3 Responsibility
Wen Chien Yen	Handle team coordination, quality control, deployment verification, README and submission notes, final emails, and contribution tracking. Confirm that the prototype works on the cloud instance and that final work is merged into the default branch.
Oscar Wang	Coordinate M3v1 document formatting, Table of Contents, UI/UX wireframes, Figma link, and frontend horizontal prototype pages/navigation. Help ensure the UI flow matches the wireframes and is easy to follow.
Banghao Yuan	Lead backend/API work and Backend Architecture. Update backend architecture based on the current implementation, finalize API routes, support signup, login, course search, and course details, and help finalize Priority 1 and Priority 2 requirements.
Sean Wang	Lead Database Architecture. Update data definitions if needed, revise the ERD from M2v2, create or enhance the EER diagram, update schema.sql and seed.sql, and make sure the database supports Priority 1 requirements.
Enoch Lin	Lead GitHub organization and software review preparation. Manage branches and merges, make sure final work is in the default branch, keep `.env` out of GitHub, update `.env.example`, and check code structure and commit quality.
Kashif Zada	Lead security, validation, and QA support. Review credential handling, access control, frontend validation, error messages, known issues, and help test the horizontal prototype for usability and consistency.

Team Contributions

Team lead assigned contribution scores. All team members have submitted confidential feedback via the instructor provided Google form.

Team Member	Wen Chien Yen	Oscar Wang	Banghao Yuan	Sean Wang	Enoch Lin	Kashif Zada
Contribution Score	10	10	10	10	10	10

Team Contributions

Wen-Chien Yen contributed to Milestone 3 as the Team Lead and Scrum Master. I coordinated team responsibilities, monitored checkpoint requirements, and helped keep the team focused on the required Milestone 3 deliverables. I reviewed the Milestone 3 README, clarified the required checkpoint structure, and assigned responsibilities to each team member based on their project roles. I also worked on deployment verification and prototype quality control. I helped troubleshoot the deployed prototype, including the frontend-backend connection, Nginx API routing, EC2 backend configuration, and the RDS database mismatch. I verified that the deployed application connected to the correct `better_student_center` database, confirmed that signup and course search worked through the public AWS URL, and updated the credentials instructions so the instructor could access the correct database and prototype environment. In addition, I prepared team communication, organized next-step workflows, reviewed README and submission requirements.

Oscar Wang contributed to Milestone 3 by working on both document formatting and frontend structure. For the documentation side, he reviewed and formatted the Milestone 3 document and updated the table of contents so the final document was easier to follow and aligned with the required structure. For frontend development, Oscar significantly improved the UI skeleton and modernized the site layout. He redesigned the vertical prototype to use a navigation bar so users could move more easily between major sections such as Home, Course Search, Login, and Signup. He also changed the navigation structure to use React Router instead of the earlier manual view-mapping approach, making the frontend structure more scalable and closer to standard React development practices. Oscar also helped establish the foundation for conditional page access, which supports the idea that different users should only access pages appropriate to their role. His work improved the usability, structure, and maintainability of the frontend for the horizontal prototype.

Banghao Yuan contributed to Milestone 3 by leading backend architecture updates and system design improvements. He updated the backend architecture section to better reflect the current implementation of the project, including backend modules, security, validation, correctness handling, and scalability. These updates helped explain how the backend supports the current prototype and how it can be expanded in later development. Banghao also updated the UML diagram to match the current system design and implementation. This helped connect the documentation to the actual project structure, including frontend modules, backend API routes, database interactions, and the overall system flow. His work helped make the backend architecture more complete, explainable, and consistent with the implemented prototype.

Sean Wang contributed to Milestone 3 through database architecture, frontend/backend integration, and feature refinement. For the database portion, he reused and refined the data definitions from Milestone 2, revised the high-level functional requirements based on team feedback, and added an EER diagram that models all database tables in the application. He also added a normalization section to the database design, helping show that the schema follows good relational database design practices. Sean also expanded class section data to include professor and contact information. This improved the course search feature by making course results more informative and useful for users. He also updated the login/signup flow and improved course search styling. On the frontend and integration side, Sean added Campus Map files to the frontend, made the class search interface more compact, made each course collapsible, and allowed sections to be selectable. He also connected the Campus Map to the Course Search page, helping support one of the project's key planned features: showing class locations through the student schedule/course planning workflow.

Enoch Lin contributed to Milestone 3 as the GitHub Master and repository support member. His role focused on helping the team maintain the project repository, organize code changes, and support the team's Git workflow. This included helping ensure that team members worked from the correct branch, that changes were pushed and merged properly, and that the final work was prepared for the default branch used for grading. For Milestone 3, Enoch's contribution supported the software review expectations by helping the team keep the repository organized and ready for integration. Since Milestone 3 includes evaluation of GitHub organization, meaningful commits, branch usage, and final default-branch readiness, his role was important for keeping the team's codebase manageable and submission-ready. Enoch also supported the team by helping with integration awareness and making sure repository work aligned with the team's deployment and prototype testing needs.

Kashif Zada contributed to Milestone 3 by supporting security, validation, and quality assurance for the prototype. His assigned focus was to review credential handling, access control, frontend validation, error messages, known issues, and prototype usability. This work supported the Milestone 3 requirement that the project demonstrate basic security practices and engineering discipline, not just feature count. Kashif also helped test the prototype from both local and deployed environments. He reported issues with server-side login behavior, which helped the team identify a frontend/backend field-name mismatch in the login request. This testing helped confirm that some issues were not caused by AWS, Nginx, or the database, but by frontend request body formatting. His work helped improve

the team’s understanding of user-facing errors, validation behavior, and the difference between local testing and deployed server testing. Kashif’s QA feedback helped the team identify problems that an actual client or instructor might experience when testing the application.